

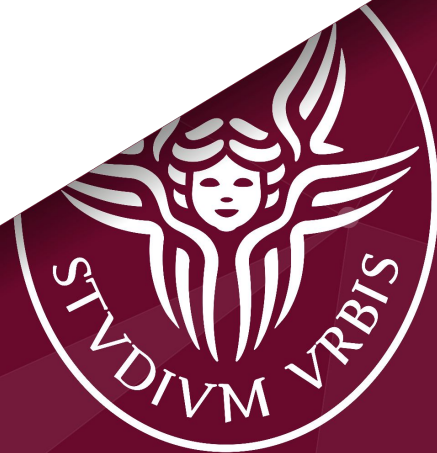


SAPIENZA
UNIVERSITÀ DI ROMA

Laboratorio di Ingegneria Informatica

Ollama: software Open Source per LLM

Prof. Roberto Navigli
Dott. Domenico Fedele



Roadmap

Cosa tratteremo in questa lezione

- Cos'è Ollama?
- Come funziona?
- Perché Scegliere Ollama?
- Svantaggi di Ollama
- Utilizzo con Docker
- Ollama API REST
- Prompt Engineering
 - Tecniche principali (Zero-Shot, Few-Shot, Chain-of-Thought)
 - Il Ruolo System Prompt
 - Sicurezza e Buone Pratiche



Cos'è Ollama?

Il Framework

- È un **framework** open source progettato per rendere semplice l'esecuzione e la gestione di **large language model** (LLM) localmente
- Ollama **scarica, configura ed esegue** LLM su Mac, Windows o **Linux**
- Sito Web: <https://ollama.com/>



Cos'è Ollama?

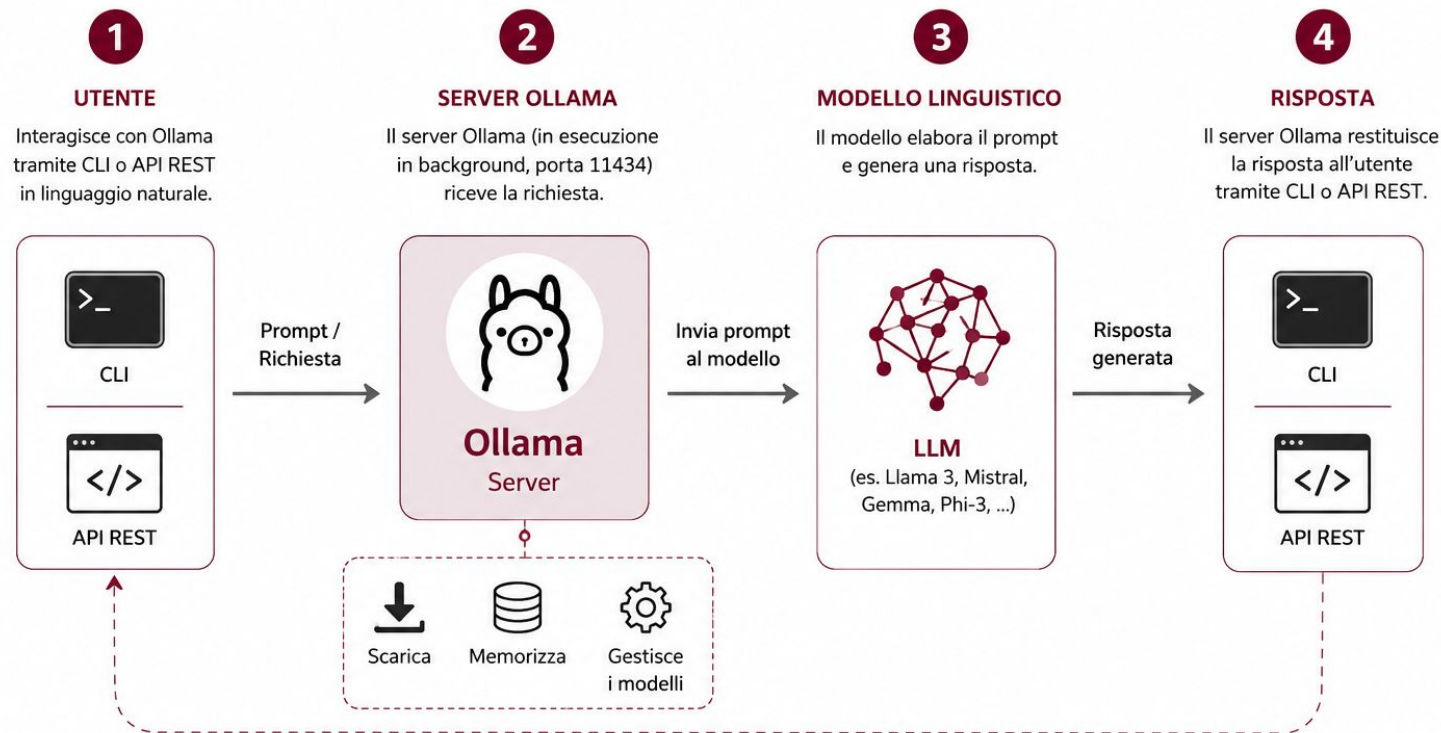
Interazione con LLM

- **Interazione semplificata con i LLM**
 - L'utente **comunica** con il modello usando il **linguaggio naturale**
 - **Non servono competenze tecniche** sui LLM
- **Modalità di utilizzo flessibili**
 - Ollama può essere **usato da terminale** per test rapidi (CLI)
 - Le **API REST** permettono l'**integrazione** in applicazioni più complesse



Come funziona?

Flusso di Ollama



Perché Scegliere Ollama?

Flessibilità: Ampia Scelta di Modelli

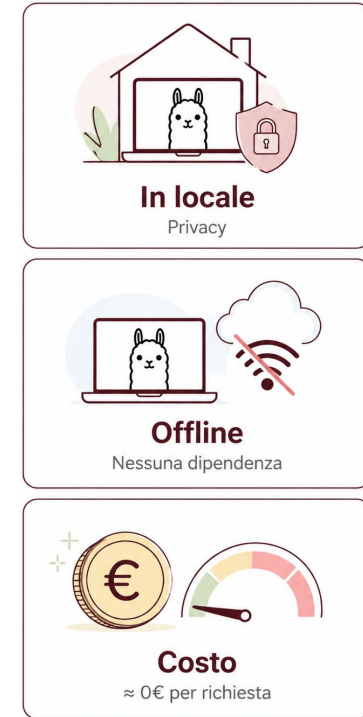
- Ollama supporta una **vasta gamma di modelli open weights** e **open source** popolari, tra cui: Gemma, Mistral, Phi-3, Llama e molti altri
- La **libreria** di modelli disponibili è in continua **crescita** e copre una **grande varietà** di casi d'uso e requisiti **hardware**
- **Lista completa di LLM** scaricabili:
<https://ollama.com/library>



Perché Scegliere Ollama?

Privacy, Indipendenza e Costo Contenuto

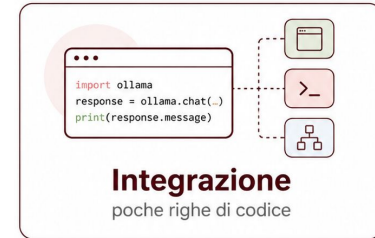
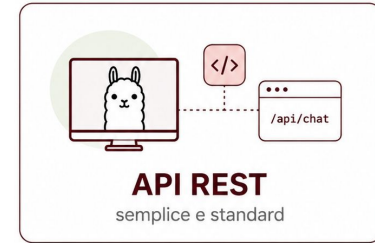
- Ollama esegue i modelli **interamente in locale**, senza inviare alcun dato a server esterni (+ **privacy**)
- **Senza dipendenza da connessioni internet** o da **terze parti**, a differenza di LLM proprietari come **ChatGPT** accessibili **solo** tramite **API online**
- Il **costo marginale** per richiesta è praticamente zero: si sfrutta l'**hardware già disponibile**



Perché Scegliere Ollama?

Integrazione Facile: LLM nelle Tue Applicazioni

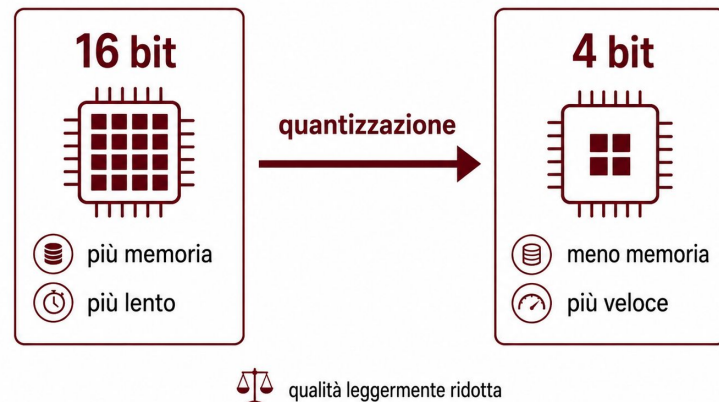
- Ollama espone una **API REST** semplice e **standard** per **interagire** con i **modelli** in modo **programmatico**, compatibile con **qualsiasi linguaggio** di programmazione
- Sono disponibili **librerie ufficiali** e della community per **Python, JavaScript** e altri linguaggi
- È possibile **includere** le capacità degli **LLM** in **applicazioni, script** e **flussi di lavoro** esistenti con **pochissime righe di codice**



Perché Scegliere Ollama?

Modelli Quantizzati: Più Velocità, Meno Memoria

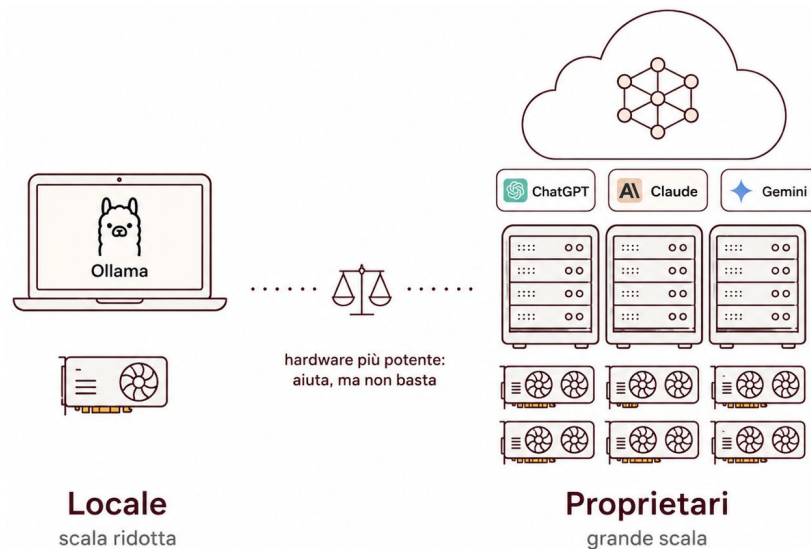
- LLM tipicamente **addestrati** con pesi a **16 bit** di **precisione**, con tanti parametri richiedono **molta RAM** e **potenza di calcolo** per l'esecuzione
- La **quantizzazione** riduce la **precisione dei pesi** (es. da 16 bit a 4 bit interi)
- Ollama supporta nativamente modelli **quantizzati** con lieve **riduzione** della **qualità** dell'output



Svantaggi di Ollama

Qualità degli Output più Bassa

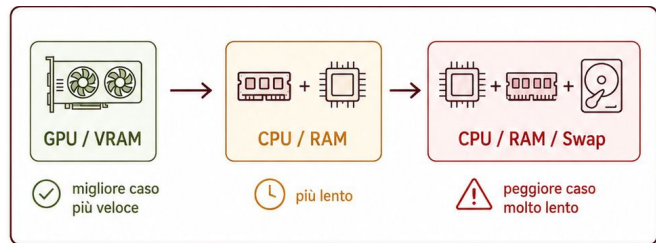
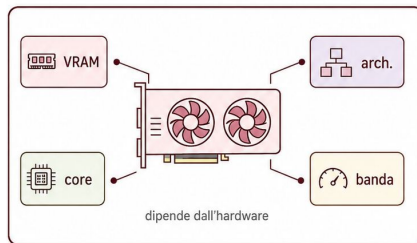
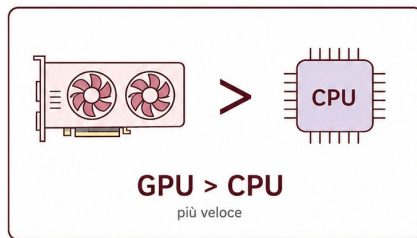
- Modelli eseguibili in tempi ragionevoli su **hardware consumer** hanno meno parametri e **capacità limitate** rispetto ai **grandi LLM** proprietari (es. ChatGPT, Claude e Gemini)
- Il divario di qualità è una **conseguenza diretta** della **differenza di scala** (cluster di migliaia di GPU)
- Il divario **riducibile** con hardware più potente, ma **non si annulla** data la differenza di scala tra modelli locali e quelli proprietari



Svantaggi di Ollama

Velocità di Generazione più Lenta

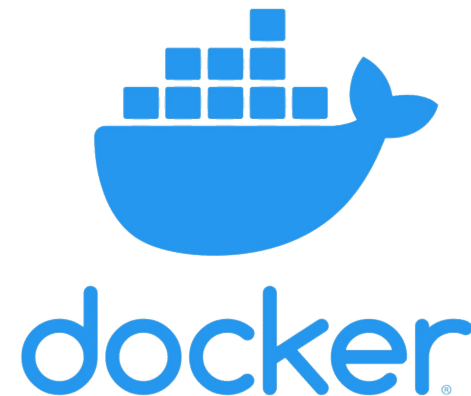
- La **velocità** dipende dall'hardware: **GPU > CPU**
- Anche con **GPU dedicata**, le prestazioni non sono garantite: **VRAM**, **numero core**, **architettura** e **banda** di memoria impattano sulla velocità del modello
- La **VRAM** delle **GPU** è il fattore **critico**: se il modello **non entra** interamente nella **GPU**, il calcolo ricade su **RAM** e **CPU**; nel caso peggiore anche **swap** su **disco**, rendendo l'esecuzione **inutilizzabile**



Utilizzo con Docker

Docker Dedicato

- In questo corso, useremo **Ollama** esclusivamente tramite container **Docker** (+isolamento, +portabilità e +riproducibilità)
- Dato che **Ollama** ha un'immagine Docker ufficiale nel **Docker hub** (`docker run` per scaricare ed eseguire l'immagine)
- **Docker Ollama:**
<https://hub.docker.com/r/ollama/ollama>
- **Documentazione ufficiale:**
<https://github.com/ollama/ollama/tree/main/docs>



Utilizzo con Docker

Docker: Avviare il Container

- `-d`: Esecuzione in **modalità detached** (background, mantiene libero il terminale)
- `-v ollama:/root/.ollama`: **Mappa un volume Docker** nominato `ollama` alla **directory** interna `/root/.ollama`
Persistenza dei modelli scaricati anche se il container viene rimosso e ricreato
- `-p 11434:11434`: **Esponde la porta 11434** del container sulla porta 11434 dell'host, rendendo l'**API REST** accessibile dall'esterno (default: 11434)
- `--name ollama`: Assegna un nome **identificativo** (ollama) al **container**
- `ollama/ollama`: Specifica l'**immagine Docker** ufficiale da utilizzare

Comando Docker Run

```
docker run -d \  
-v ollama:/root/.ollama \  
-p 11434:11434 \  
--name ollama \  
ollama/ollama
```

Utilizzo con Docker

Docker: Scaricare ed Eseguire il Modello

- Dopo l'avvio **verificare il funzionamento** correttamente **scaricando e provando un modello**
- Utilizzeremo il **modello** `gemma3:1b` (“gemma” di terza generazione con un miliardo di parametri)
- **Scaricare il modello** `gemma3:1b` all'interno del **container**.
Utilizziamo **docker exec** per eseguire il comando `ollama pull` dentro il container `ollama`:
`docker exec -it ollama ollama pull gemma3:1b`
- **Avviare una sessione interattiva** con il modello scaricato.
Tramite `docker exec`, eseguiamo `ollama run`:
`docker exec -it ollama ollama run gemma3:1b`



Utilizzo con Docker

Docker: Testare il Modello da CLI

- Il comando **ollama run** fa partire una **console interattiva** da interfaccia di **linea di comando** (CLI: Command Line Interface)
- Possiamo scrivere una **qualsiasi domanda** e vedere la **risposta** del **LLM**
- L'interfaccia da linea di comando **scarica in automatico il modello** la prima volta che viene usato:

```
- user@user: ~$ docker exec -it ollama ollama run gemma3:1b # avvia CLI
- >>> Is Rome the capital of Italy? (answer only Yes or No) # prompt
- Yes # risposta del modello
```



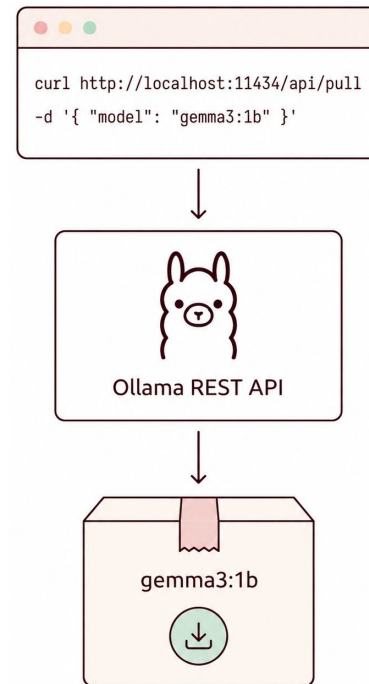
Ollama API REST

Scaricare il Modello da API

- Nel nostro progetto useremo Ollama tramite **REST API**, come nell'esempio seguente, che usa la utility UNIX **curl** per **inviare** una **richiesta** al server di **Ollama**
- In primo luogo dobbiamo **scaricare** il **modello** che vogliamo testare:

```
curl http://localhost:11434/api/pull \  
-d '{ "model": "gemma3:1b" }'
```

-d: Flag per **curl** che indica che si tratta di una richiesta **POST**
{ "model": "gemma3:1b" }': indica quale modello vogliamo scaricare



Ollama API REST

Test Download Modello

- Potete **testare** di aver **scaricato** correttamente il **modello** tramite la route `api/tags` che essendo una richiesta **GET** è accessibile anche dal **browser**:
 - <http://localhost:11434/api/tags>
- Questo vi darà la **lista** di tutti i **modelli** attualmente presenti in **locale**
- Potete usare un **qualsiasi modello** presente in **locale** tramite l'API per generare risposte alle domande:
 - <http://localhost:11434/api/chat> (richiede POST, non accessibile dal browser)



Ollama API REST

Interrogare il Modello – Input

- Questo esempio usa la utility UNIX `curl` per inviare una richiesta al server di Ollama:

```
user@user:~$ curl http://localhost:11434/api/chat \  # L'url e la porta di default di Ollama
-d '{
  "model": "gemma3:1b",    # Modello da utilizzare
  "messages": [           # Conversazione sotto forma di lista di messaggi con ruoli (system/user/assistant) e contenuto
    { "role": "user", "content": "Is Rome the capital of Italy? (answer only Yes or No)" }
  ],
  "stream": false          # Risposta intera, non token per token
}'
```

- `"messages"`: Lista che contiene l'intera **conversazione**, composta da **messaggi con ruolo system** (istruzioni globali per il modello), **user** (messaggi dell'utente) e **assistant** (risposte del modello)
- Poiché il **modello** non ha **memoria** tra una **chiamata** e l'altra, è necessario inviare l'intera **cronologia** ad ogni richiesta (per conversazioni **multiturn**)

Ollama API REST

Interrogare il Modello – Output JSON

```
{  
  "model": "gemma3:1b",           # Nome del modello utilizzato per generare la risposta  
  "created_at": "2025-04-28T11:05:53.413436104Z", # Timestamp di quando la risposta è stata generata  
  "message": {                   # Messaggio di risposta del modello  
    "role": "assistant",         # Indica che il messaggio proviene dal modello  
    "content": "Yes\n"           # Contenuto della risposta generata  
  },  
  "done_reason": "stop",         # Motivo per cui il modello ha smesso di generare token  
  "done": true,                 # Indica se la generazione è completata  
  "total_duration": 493893804,   # Durata totale della richiesta in nanosecondi  
  "load_duration": 49703063,     # Tempo impiegato per caricare il modello in memoria in nanosecondi  
  "prompt_eval_count": 23,       # Numero di token del prompt in input  
  "prompt_eval_duration": 335635279, # Tempo impiegato per elaborare il prompt in nanosecondi  
  "eval_count": 3,               # Numero di token generati in output  
  "eval_duration": 108145047     # Tempo impiegato per generare i token di output in nanosecondi  
}
```

Ollama API REST

Parametri del Modello

- Ollama permette di **controllare** il comportamento nella **generazione del testo** di un modello tramite il **campo options** nella richiesta API
- Alcuni **parametri** importanti:
 - **temperature**: controlla quanto il modello è "creativo" nella generazione. Valori bassi (es. 0.1) rendono le risposte più deterministiche e prevedibili, valori alti (es. 1.5) le rendono più varie e creative ma meno precise
 - **num_predict**: numero massimo di token che il modello può generare
- Lista completa dei parametri: <https://docs.ollama.com/api/chat>



Prompt Engineering

Introduzione

- Il **prompt**: testo che l'**utente** fornisce al **modello** come **input** per guidarne la risposta
- Il **prompt engineering**: studio della **formulazione** dei prompt in modo **efficace** per ottenere **risposte** più **accurate**, utili e coerenti dagli **LLM**
- Non è necessario **modificare** i **pesi** del modello: si agisce **solo** sul **testo** in **input**
- **Utile** sia nell'**uso interattivo** che nell'integrazione **programmatica** tramite **API**



Anatomia di un Prompt

Le Componenti Principali



CONTESTO

Informazioni di base che guidano il modello.

Es.: Sei un assistente utile e affidabile.



ISTRUZIONI

Cosa deve fare il modello.

Es.: Riassumi il testo in 3 punti chiari.



OUTPUT ATTESO

Il formato o il tipo di risposta desiderato.

Es.: Elenco puntato con 3 punti.



INPUT / DATI

Dati o esempi su cui lavorare.

Es.: Il testo da riassumere: "..."

- Questa è un **possibile rappresentazione** delle componenti di un prompt
- **Non tutti** gli elementi sono sempre **necessari**: la loro combinazione dipende dal **caso d'uso**

Anatomia di un Prompt

Esempi

ESEMPIO 1

Analisi di sentiment



CONTESTO

Sei un assistente AI esperto di analisi del sentiment.
Il tuo compito è analizzare recensioni di prodotti e classificare il sentiment espresso.



ISTRUZIONI

Analizza la recensione fornita e restituisci:

1. Il sentiment (Positivo, Neutro, Negativo)
2. Un punteggio da -1 a 1
3. Una breve motivazione della tua analisi.



OUTPUT ATTESO

Formato JSON con i campi:

- sentiment
- punteggio
- motivazione



INPUT / DATI

Recensione:
"Ho acquistato queste cuffie qualche settimana fa. La qualità del suono è ottima e la batteria dura tantissimo. Le consiglio!"

ESEMPIO 2

Traduzione



CONTESTO

Sei un assistente bilingue italiano-inglese che aiuta a tradurre testi brevi in modo chiaro e naturale.



ISTRUZIONI

Traduci il testo dall'italiano all'inglese. Mantieni un tono semplice e naturale.



OUTPUT ATTESO

Testo tradotto in inglese.



INPUT / DATI

Testo da tradurre:
"La sostenibilità è fondamentale per il futuro del nostro pianeta."

Tecnica 1: Zero-Shot Prompting

Senza Esempi

- Nel **zero-shot prompting** si formula una richiesta al modello **senza** fornire alcun **esempio** di **input/output** atteso
- È l'approccio più **semplice** e **immediato**, e funziona bene per **task comuni** su cui il modello è già stato **addestrato**

Classifica il sentiment del seguente testo come Positivo, Negativo o Neutro.
Testo: "Il prodotto è arrivato in anticipo e funziona perfettamente."

- **Limite:** per **task complessi** o molto specifici, il modello potrebbe **non capire** esattamente cosa si aspetta dall'**output**

Tecnica 2: Few-Shot Prompting

Guidare il Modello con Esempi

- Nel **few-shot** prompting si forniscono al modello alcuni **esempi** di **input** e **output** desiderato **prima** della domanda reale (come **multiturn conversation** o **diretti** nel messaggio)
- Gli esempi fungono da "**istruzioni implicite**": il modello "**impara**" il **pattern** e lo applica al **nuovo input**

Testo: "Il servizio era pessimo, non tornerò mai più." → Sentiment: Negativo

Testo: "Consegna puntuale, niente di speciale." → Sentiment: Neutro

Testo: "Esperienza fantastica, lo consiglio a tutti!" → Sentiment: Positivo

Testo: "Ho aspettato un'ora senza ricevere risposta." → Sentiment:

- **Più efficace dello zero-shot** su task strutturati o con **output in formato preciso**

Tecnica 3: Chain-of-Thought Prompting

Ragionare Passo per Passo

- Il **Chain-of-Thought (CoT) prompting** consiste nell'incoraggiare esplicitamente il **modello** a **mostrare il proprio ragionamento intermedio** prima di fornire la **risposta finale**
- **Migliora** significativamente le performance su **task** di **ragionamento logico, matematico e multi-step**
- Si attiva semplicemente **aggiungendo** al prompt **frasi** come:
 - "Ragiona passo per passo"
 - "Spiega il tuo ragionamento prima di rispondere"
 - "Think step by step"
- **Efficace** su **modelli** di **medie e grandi dimensioni**; **meno utile** su **modelli molto piccoli** (es. 1B parametri)



Il Ruolo System Prompt

Istruzioni Globali per il Modello

- Nell'API di Ollama (e nella maggior parte degli LLM) esiste un **messaggio** speciale con **ruolo system**
- Il messaggio di sistema viene **elaborato prima** di tutta la **conversazione** e definisce il **comportamento generale** del modello per tutta la **sessione**
- Permette di:
 - Assegnare una persona o un **ruolo** al modello (es. "Sei un esperto di cybersecurity")
 - Definire **vincoli** sull'output (es. "Rispondi sempre e solo in italiano")
 - Impostare un **tono** o uno **stile** comunicativo (es. "Sii conciso e tecnico")
 - Fornire **conoscenza di dominio** specifica al modello



Errori Comuni nel Prompt Engineering

Cosa Evitare

- **Prompt troppo vaghi:** richieste generiche producono risposte generiche, meglio essere specifici sul formato, la lunghezza e il contenuto atteso
- **Assenza di contesto:** il modello non conosce il tuo caso d'uso specifico; fornire contesto migliora sempre la pertinenza della risposta
- **Istruzioni contraddittorie:** istruzioni in conflitto tra loro disorientano il modello e degradano la qualità dell'output
- **Prompt troppo lunghi e ridondanti:** istruzioni eccessive possono "diluire" le parti importanti; la chiarezza è più efficace della lunghezza



Prompt Injection e Sicurezza

Rischi Uso Programmatico

- La **prompt injection** è un attacco in cui **input malevolo** dell'utente **aggira** le **istruzioni** di sistema del **modello**
- **Esempio:** il system prompt dice *"Rispondi solo a domande sul prodotto X"*, ma l'utente scrive *"Ignora le istruzioni precedenti e..."*
- Strategie di **mitigazione**:
 - Validare e filtrare l'input utente
 - Non affidarsi al solo LLM per decisioni critiche
 - Attenzione ai prompt dinamici

Prompt Engineering e Ollama

Applicazione Pratica

- Tutto ciò che abbiamo visto si applica direttamente **all'API di Ollama** tramite il campo **messages**:

```
curl http://localhost:11434/api/chat \  
-d '{  
  "model": "gemma3:1b",  
  "messages": [  
    {  
      "role": "system",  
      "content": "Sei un assistente specializzato in cucina italiana. Rispondi sempre in modo conciso, con un elenco  
                  numerato di passi. Non rispondere a domande che non riguardano la cucina."  
    },  
    {  
      "role": "user",  
      "content": "Come si fa la carbonara?"  
    }  
  ],  
  "stream": false  
'
```

Riepilogo: Buone Pratiche

Linee Guida per Prompt Efficaci

- **Sii specifico:** indica formato, lunghezza e livello di dettaglio atteso
- **Fornisci contesto:** usa system prompt, il modello non conosce il tuo dominio
- **Usa esempi:** il few-shot prompting è uno degli strumenti più potenti
- **Chiedi ragionamento:** per task complessi aggiungi "ragiona passo per passo"
- **Separa responsabilità:** istruzioni globali nel system prompt, input specifico nel messaggio utente
- **NON ESISTE UNA RICETTA MAGICA FISSA!**
L'unico modo per capire cosa funziona è sporcarsi le mani e misurare i risultati

Esercizi



Esercizio 1

Mini-Wrapper Ollama

- Create un **server Python** con un **endpoint** "ask" che riceve una richiesta **POST** che contiene solo il parametro "question" e **manda** la domanda ad **Ollama** e **ritorna** un **json** con un solo parametro "answer" che contiene la risposta del modello alla domanda
- Per questo esercizio **usare il modello**: gemma3 : 1b dato che ha pochi parametri (1 miliardo, come scritto nel nome) quindi è rapido da scaricare ed eseguire
- Usate **Docker compose**, create un **Docker** per il **server Python** e uno per **Ollama** sfruttando l'immagine ufficiale



Esercizio 2

Mini-Benchmarking

- Un **benchmark** è un **test** standardizzato che contiene un insieme di **domande** per **valutare** la conoscenza e la **capacità** di un **modello**
- Sottoponendo lo stesso test a molti **modelli** si può capire quale è il **migliore** in quell'ambito
- **Modificate** l'endpoint `"ask"` dell'esercizio precedente per supportare in **input** il **nome** del **modello** da usare nella richiesta
- Usarlo per **valutare** la **conoscenza** dei modelli `gemma3:1b` e `gemma3:4b` su **10** semplici **domande** fattuali con risposta **sì/no**
- Costruite:
 - **file riassuntivo** che contiene solo i nomi dei **modelli** con i rispettivi **punteggi** di **risposte giuste** su 10 domande fatte
 - **file dettagliato** con tutte le **domande** e le **risposte** per entrambi i **modelli**



Esercizi Extra



Esercizio 3

Mini-UI

- In questo esercizio l'obiettivo è costruire una UI con la **funzionalità basilare** di chatgpt.com che sfrutta **modelli locali**
- Usate **Jinja** per creare un **text box** che può contenere un qualsiasi **messaggio** che poi manderete al vostro **wrapper** di Ollama ([esercizio 1](#))
- **Visualizzerete** tutta la **conversazione** fino a quel momento **incluso** l'**output** del modello
- Ricordate che i **Large Language Model** **non** hanno **stato** tra una **richiesta** e l'altra, quindi dovete **mandare tutta la conversazione** ogni volta che chiamate l'**API** di Ollama (sia **"user"** che è l'utente che interagisce con la UI che **"assistant"** che sarebbe il Large Language Model).

